Data Communications and Networking Fourth Edition

Forouzan

Chapter 11

Data Link Control

Partially Edited and
Presented by
Dr. Md. Abir Hossain

11-1 FRAMING

The data link layer needs to pack bits into frames, so that each frame is distinguishable from another. Our postal system practices a type of framing. The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter.

Topics discussed in this section:

Fixed-Size Framing
Variable-Size Framing

11-2 FLOW AND ERROR CONTROL

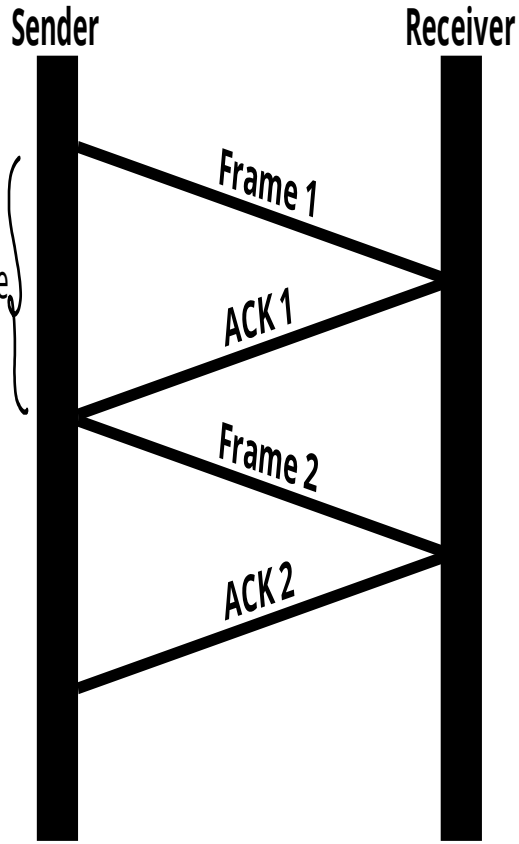
The most important responsibilities of the data link layer are flow control and error control. Collectively, these functions are known as data link control.

Topics discussed in this section:

Flow Control
Error Control

Flow Control

- Flow control is a set of procedures that tells the sender how much data it can transmit before it must wait for an acknowledgment from the receiver.
- Any receiving device has a limited speed at which it can process incoming data and a limited amount of memory in which to store incoming data.
- Incoming data must be checked and processed before they can be used. The rate of such processing is often slower than the rate of transmission.
- For this reason, each receiving device has a block of memory, called a buffer, reserved for storing incoming data until they are processed.
- If the buffer begins to fill up, the receiver must be able to tell the sender to halt transmission until it is once again able to receive.



Note

Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgment.

Error Control

- Error control is both error detection and error correction.
- It allows the receiver to inform the sender of any frames lost or damaged in transmission and coordinates the retransmission of those frames by the sender.
- In Data link layer, the term error control refers primarily to methods of error detection and retransmission.
- In data link layer error control is simply: Any time an error is detected in an exchange, specified frames are retransmitted.

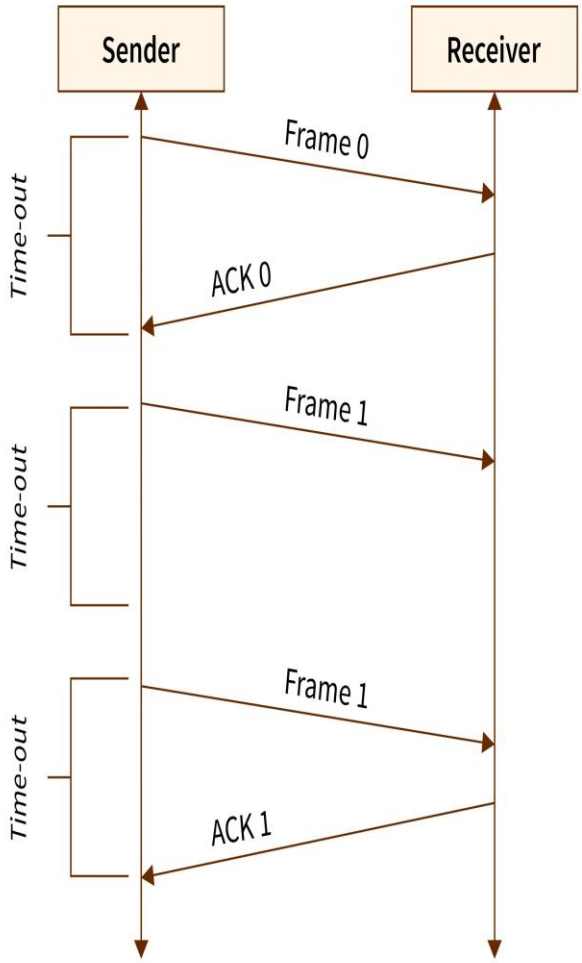


Fig: Error control in stop and wait protocol

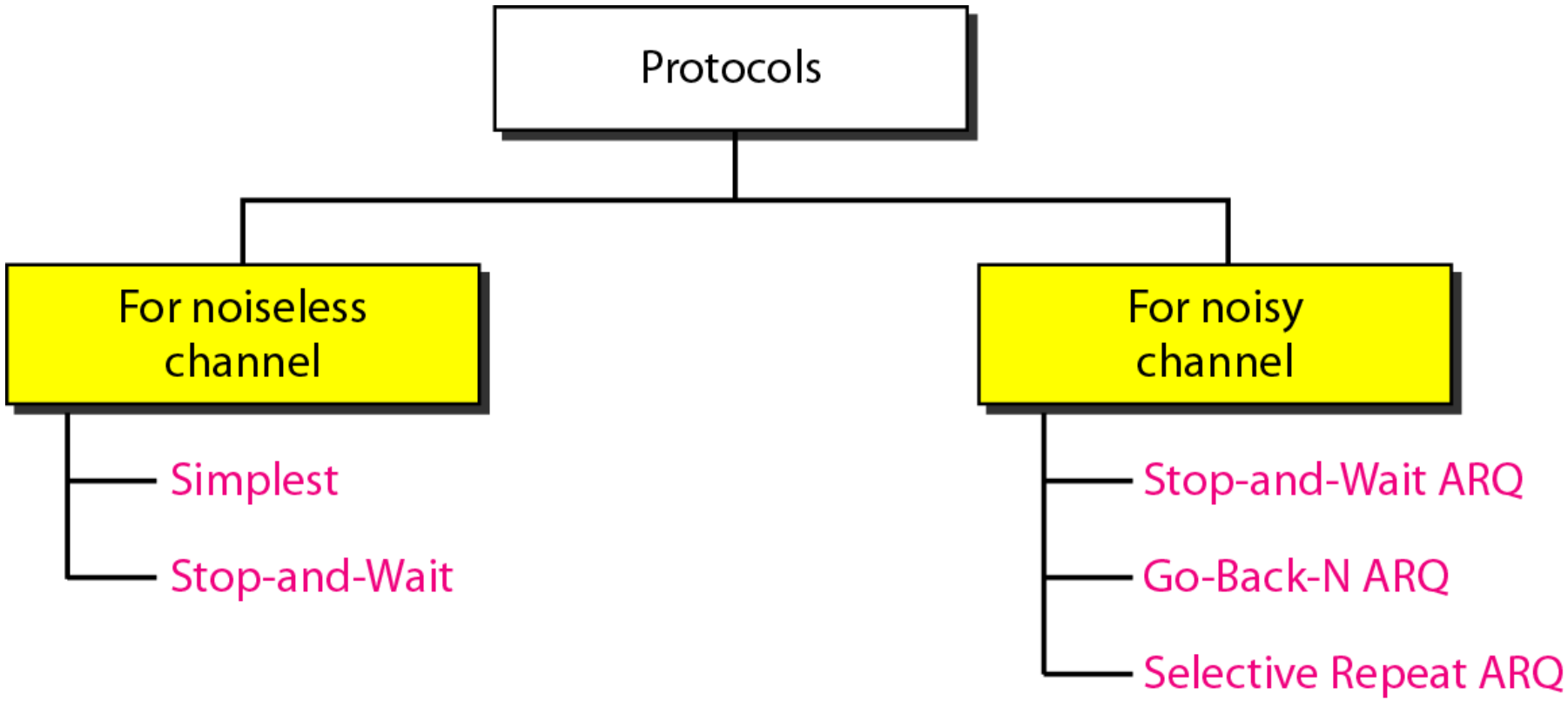
Note

Error control in the data link layer is based on automatic repeat request, which is the retransmission of data.

11-3 PROTOCOLS

Now let us see how the data link layer can combine framing, flow control, and error control to achieve the delivery of data from one node to another.

Figure 11.5 *Taxonomy of protocols discussed in this chapter*



11-4 NOISELESS CHANNELS

Let us first assume we have an ideal channel in which no frames are lost, duplicated, or corrupted. We introduce two protocols for this type of channel.

Topics discussed in this section:

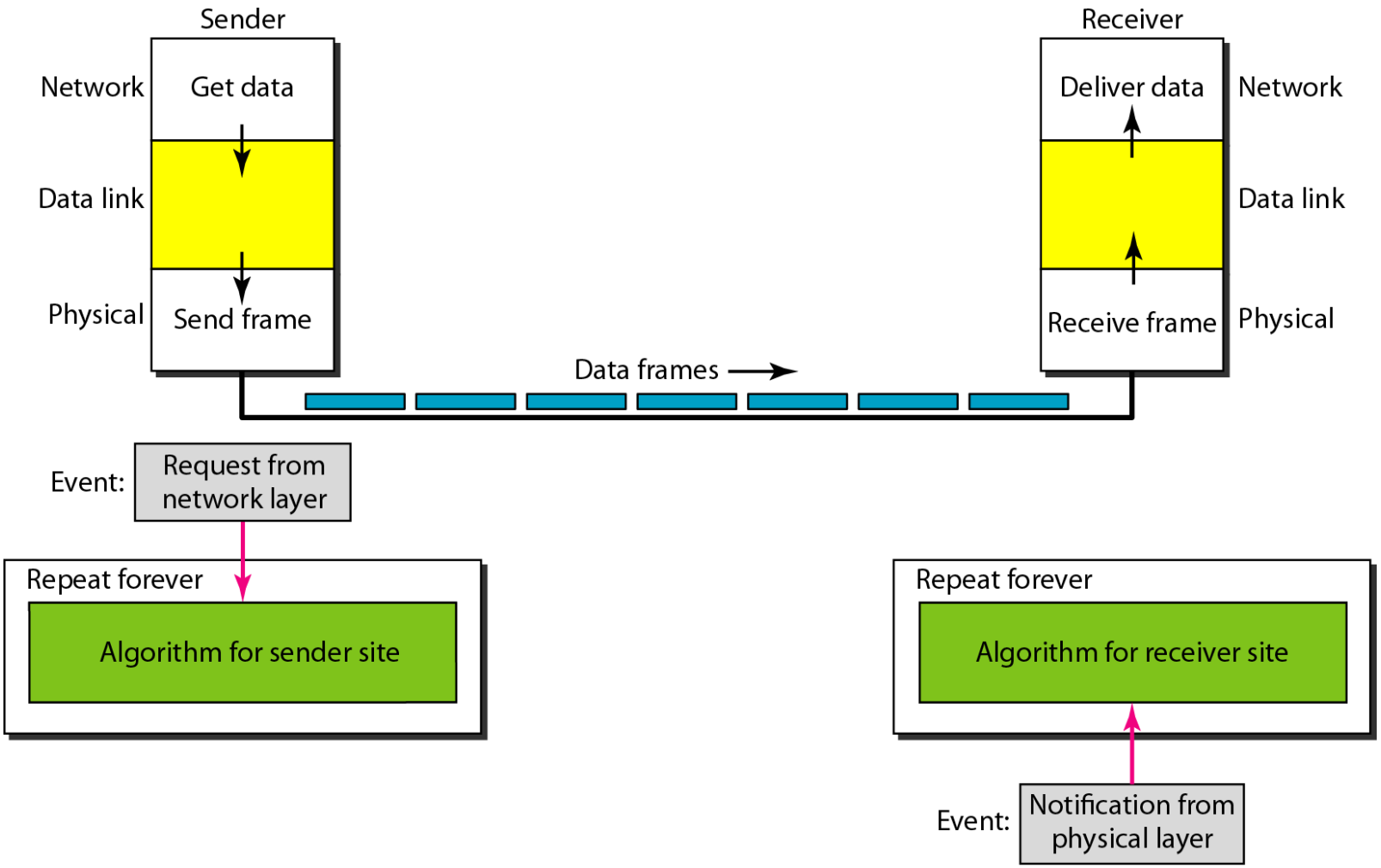
Simplest Protocol

Stop-and-Wait Protocol

Simplest Protocol

- The Simplest Protocol has no flow or error control.
- It is an unidirectional protocol in which data frames travel in only one direction, from the sender to the receiver.
- The sender site cannot send a frame until its network layer has a data packet to send.
- The data link layer at the sender site gets data from its network layer, makes a frame out of the data, and sends it.
- Similarly, the receiver site cannot deliver a data packet to its network layer until a frame arrives.
- The data link layer at the receiver site receives a frame from its physical layer, extracts data from the frame, and delivers the data to its network layer.

Figure 11.6 *The design of the simplest protocol with no flow or error control*



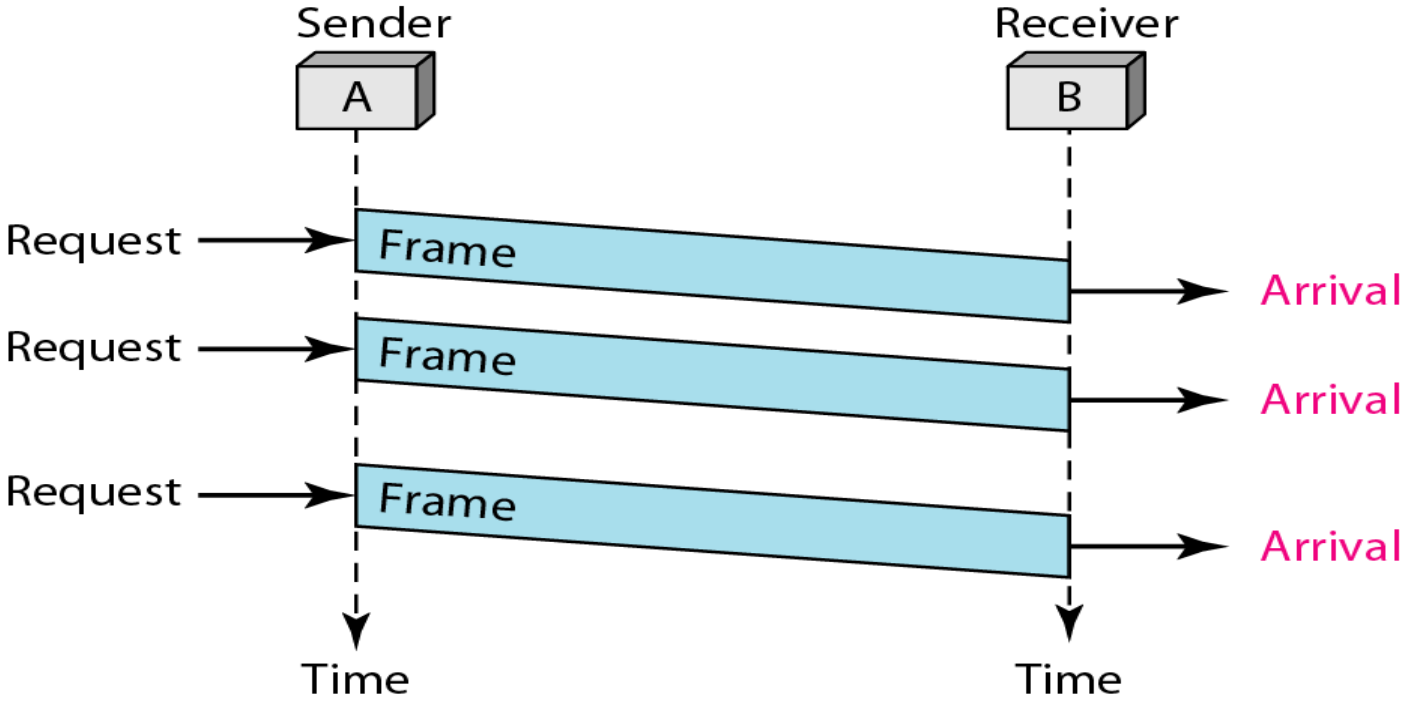
Algorithm 11.1 *Sender-site algorithm for the simplest protocol*

```
1  while(true)                          // Repeat forever
2  {
3    WaitForEvent();                    // Sleep until an event occurs
4    if(Event(RequestToSend))           //There is a packet to send
5    {
6       GetData();
7       MakeFrame();
8       SendFrame();                    //Send the frame
9    }
10 }
```

Algorithm 11.2 *Receiver-site algorithm for the simplest protocol*

```
1  while(true)                          // Repeat forever
2  {
3    WaitForEvent();                    // Sleep until an event occurs
4    if(Event(ArrivalNotification)) //Data frame arrived
5    {
6       ReceiveFrame();
7       ExtractData();
8       DeliverData();                  //Deliver data to network layer
9    }
10 }
```

Figure 11.7 *Flow diagram for Example 11.1*

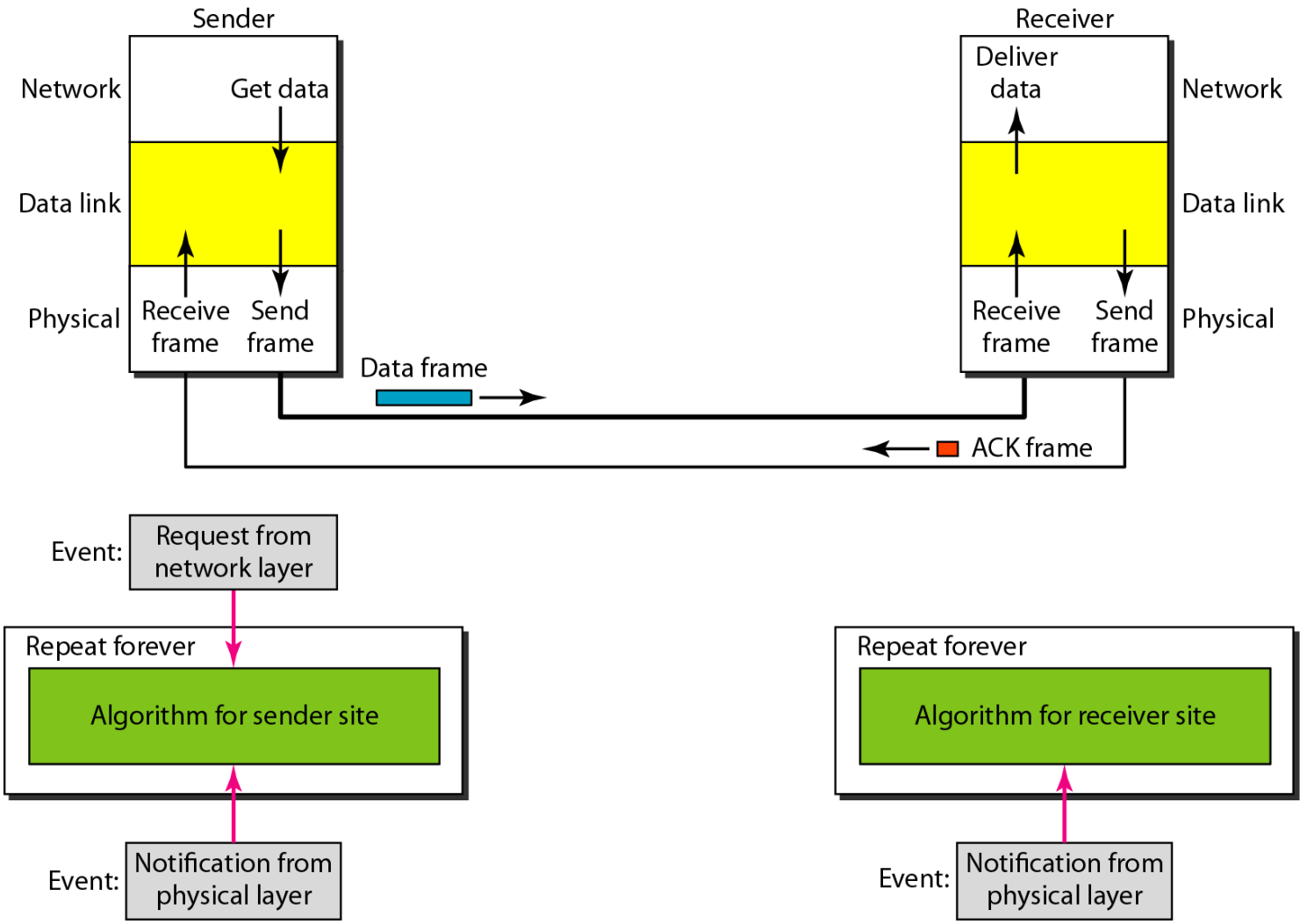


- The flow activity is very simple. The sender sends a sequence of frames without thinking about the receiver.
- To send three frames, three events occur at the sender site and three events at the receiver site.

Stop-and-wait Protocol

- In the stop-and-wait protocol, the sender sends one frame, stops until it receives confirmation from the receiver, and then sends the next frame.
- The stop-and-wait also provides unidirectional communication for data frames, but auxiliary ACK frames travel from the other direction.
- At any time, there is either one data frame on the forward channel or one ACK frame on the reverse channel.
- Here, two events can occur: a request from the network layer or an arrival notification from the physical layer.
- After a frame is sent, the algorithm must ignore another network layer request until that frame is acknowledged.

Figure 11.8 *Design of Stop-and-Wait Protocol*



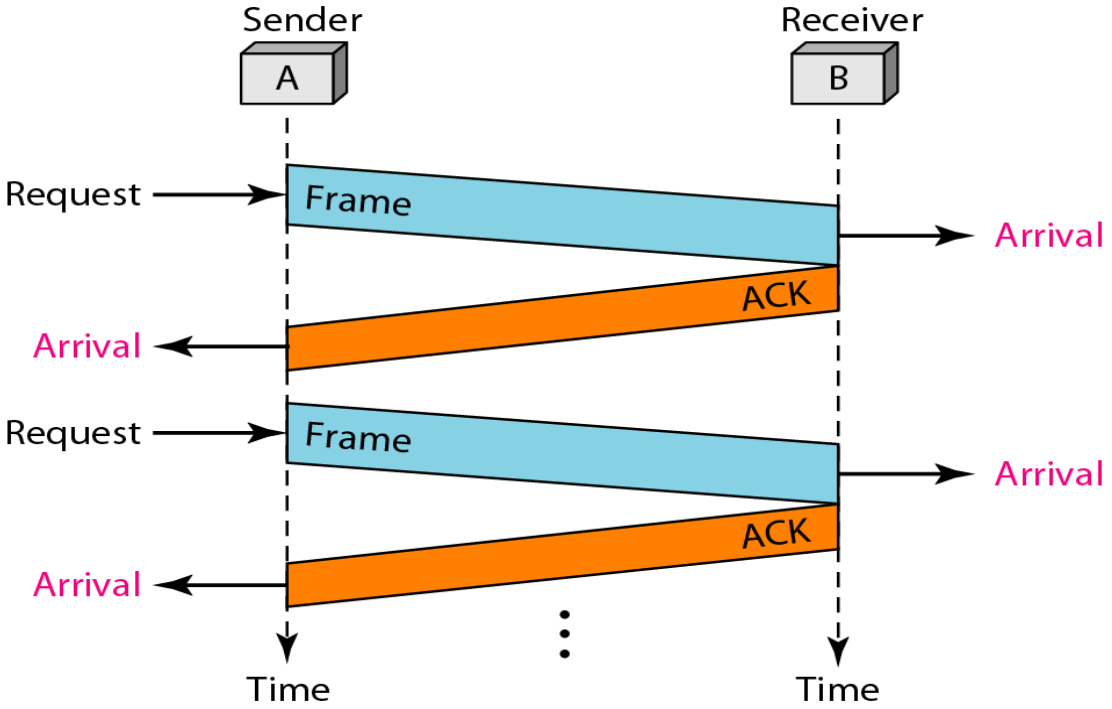
Algorithm 11.3 *Sender-site algorithm for Stop-and-Wait Protocol*

```
1  while(true)                          //Repeat forever
2  canSend = true                       //Allow the first frame to go
3  {
4    WaitForEvent();                    // Sleep until an event occurs
5    if(Event(RequestToSend) AND canSend)
6    {
7       GetData();
8       MakeFrame();
9       SendFrame();                    //Send the data frame
10      canSend = false;                //Cannot send until ACK arrives
11   }
12   WaitForEvent();                    // Sleep until an event occurs
13   if(Event(ArrivalNotification) // An ACK has arrived
14    {
15      ReceiveFrame();                 //Receive the ACK frame
16      canSend = true;
17    }
18 }
```

Algorithm 11.4 *Receiver-site algorithm for Stop-and-Wait Protocol*

```
1  while(true)                          //Repeat forever
2  {
3    WaitForEvent();                    // Sleep until an event occurs
4    if(Event(ArrivalNotification))     //Data frame arrives
5    {
6       ReceiveFrame();
7       ExtractData();
8       Deliver(data);                  //Deliver data to network layer
9       SendFrame();                    //Send an ACK frame
10   }
11 }
```

Figure 11.9 *Flow diagram for Example 11.2*



- The flow activity is quite simple and similar with simplest protocol with one exception.
- After the data frame arrives, the receiver sends an ACK frame to acknowledge the receipt and allow the sender to send the next frame.
- The sender sends one frame and waits for feedback from the receiver.
- When the ACK arrives, the sender sends the next frame.

11-5 NOISY CHANNELS

Although the Stop-and-Wait Protocol gives us an idea of how to add flow control to its predecessor, noiseless channels are nonexistent. We discuss three protocols in this section that use error control.

Topics discussed in this section:

Stop-and-Wait Automatic Repeat Request (ARQ)
Go-Back-N Automatic Repeat Request
Selective Repeat Automatic Repeat Request

Stop-and-wait ARQ Protocol

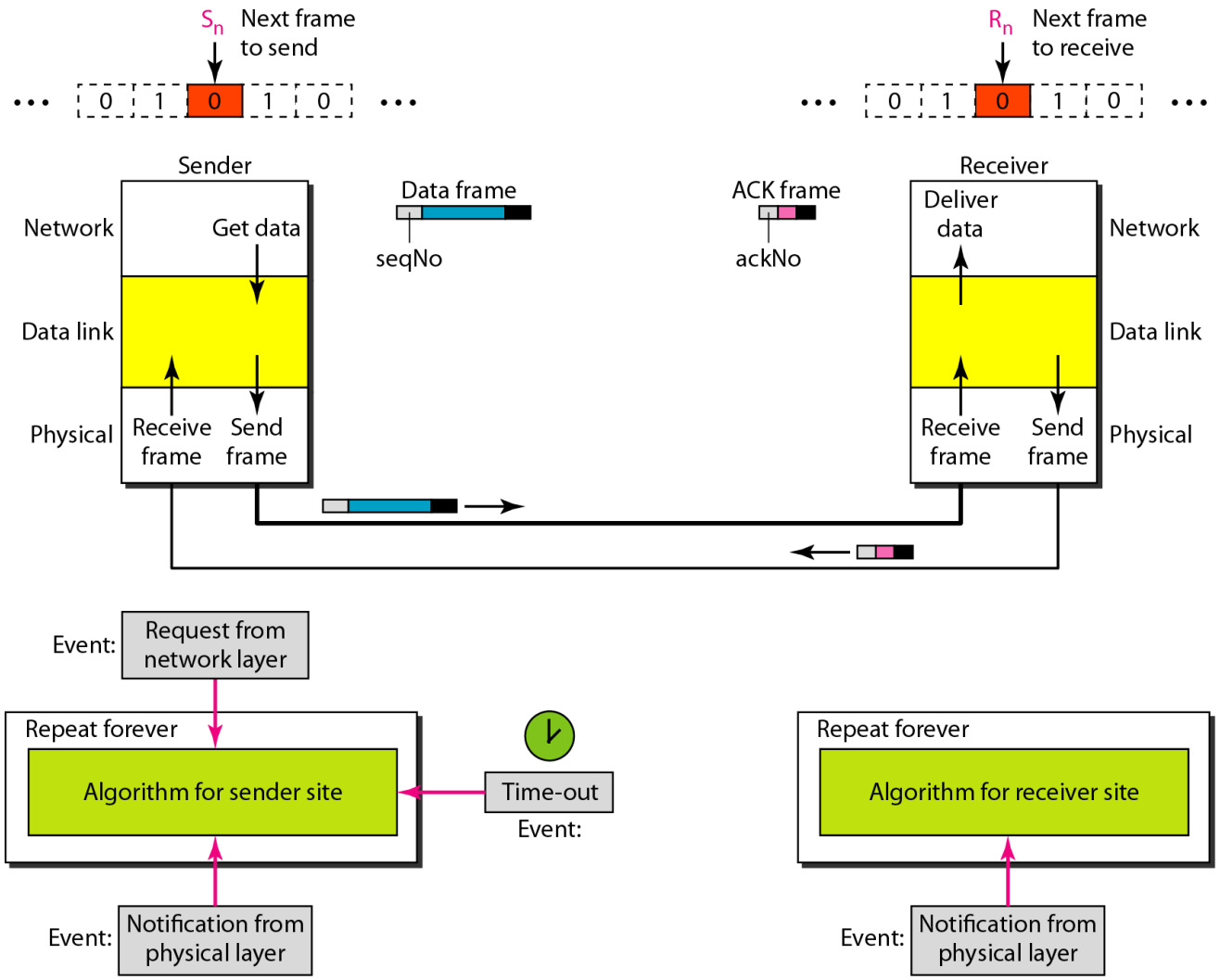
Note

Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires.

Stop-and-wait ARQ Protocol

- The stop-and-wait **A**utomatic **R**epeat re**Q**uest (stop-and-wait ARQ), protocol adds a simple error control mechanism to the stop-and-wait protocol.
- The received frame could be the correct one, a duplicated, or corrupted , or a frame out of order.
- The corrupted and lost frames need to be resent in this protocol.
- If the receiver does not respond when there is an error, how can the sender know which frame to resend?
- To solve this problem, the sender keeps a copy of the sent frame. At the same time, it starts a timer.
- If the timer expires and there is no ACK for the sent frame, the frame is resent, the copy is held, and the timer is restarted.
- Since the protocol uses the stop-and-wait mechanism, there is only one specific frame that needs an ACK even though several copies of the same frame can be in the network.

Figure 11.10 *Design of the Stop-and-Wait ARQ Protocol*



Stop-and-Wait ARQ Overview

- Sender waits "reasonable" amount of time for ACK
  - Thus Sender needs a countdown timer
  - Start the timer when a packet is sent
  - retransmits if no ACK received within the timeout period
- if pkt (or ACK) just delayed (not lost):
  - retransmission will create duplicate packet
  - Thus it requires packet sequence number and ack number to be used
  - Only two numbers are used: 0, 1

- Receiver's Ack number is what he is expected next
  - After receiving Pkt 0, sends back ACK 1
  - After receiving Pkt 1, sends back ACK 0

Algorithm 11.5 *Sender-site algorithm for Stop-and-Wait ARQ*

```
1  Sn = 0;                              // Frame 0 should be sent first
2  canSend = true;                      // Allow the first request to go
3  while(true)                          // Repeat forever
4  {
5    WaitForEvent();                    // Sleep until an event occurs
6    if(Event(RequestToSend) AND canSend)
7    {
8       GetData();
9       MakeFrame(Sn);                        //The seqNo is Sn
10      StoreFrame(Sn);                       //Keep copy
11      SendFrame(Sn);
12      StartTimer();
13      Sn = Sn + 1;
14      canSend = false;
15   }
16   WaitForEvent();                          // Sleep
```

Modulo-2 addition

(continued)

Algorithm 11.5 *Sender-site algorithm for Stop-and-Wait ARQ* *(continued)*

```
17      if(Event(ArrivalNotification)          // An ACK has arrived
18      {
19        ReceiveFrame(ackNo);                 //Receive the ACK frame
20        if(not corrupted AND ackNo == Sn)    //Valid ACK
21          {
22            Stoptimer();
23            PurgeFrame(Sn-1);                //Copy is not needed
24            canSend = true;
25          }
26       }
27
28      if(Event(TimeOut)                      // The timer expired
29      {
30       StartTimer();
31       ResendFrame(Sn-1);                    //Resend a copy check
32      }
33  }
```

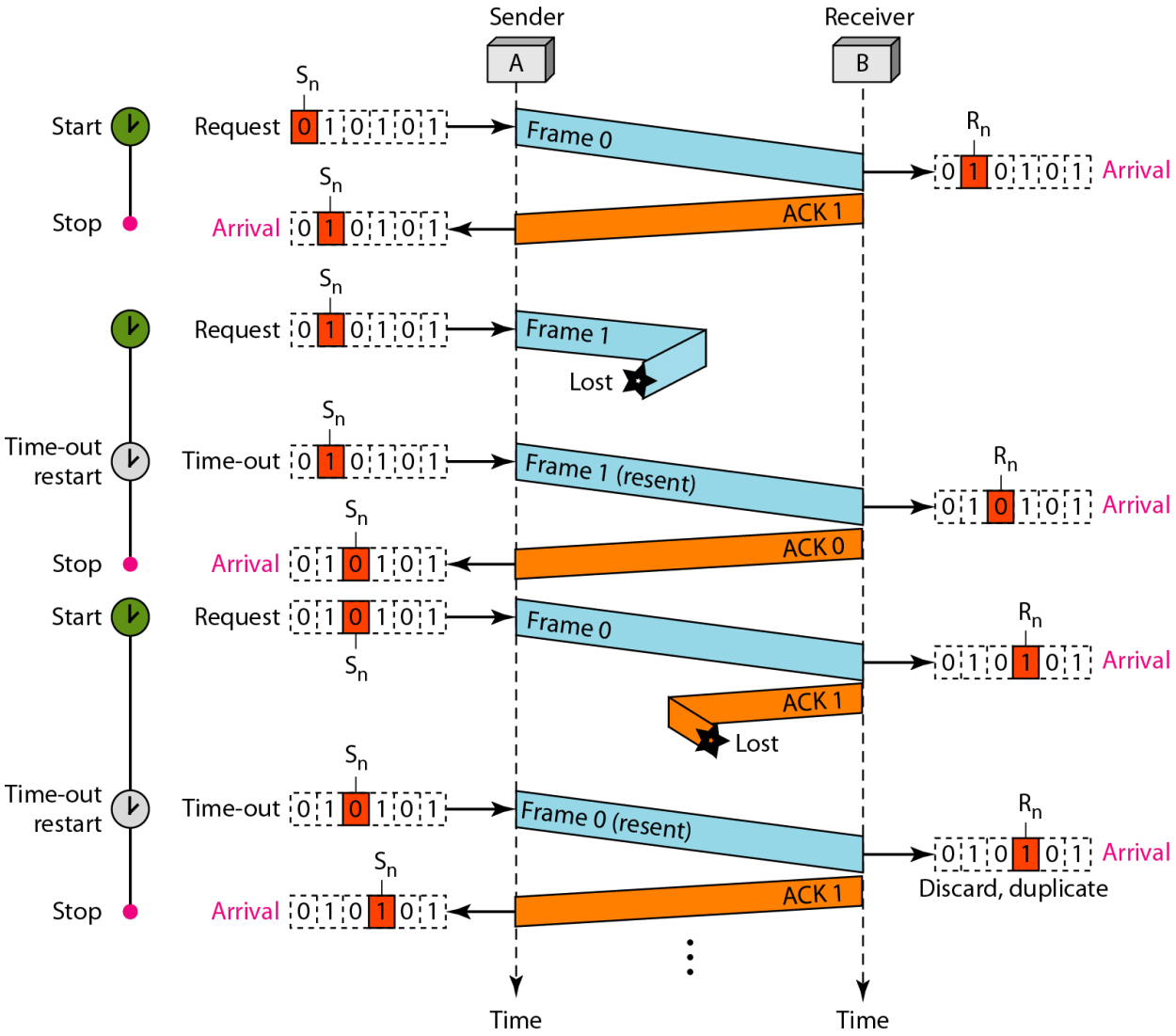
Algorithm 11.6 *Receiver-site algorithm for Stop-and-Wait ARQ Protocol*

```
1  Rn = 0;                          // Frame 0 expected to arrive first
2  while(true)
3  {
4    WaitForEvent();                // Sleep until an event occurs
5    if(Event(ArrivalNotification))  //Data frame arrives
6    {
7       ReceiveFrame();
8       if(corrupted(frame));
9          sleep();
10      if(seqNo == Rn)              //Valid data frame
11      {
12       ExtractData();
13        DeliverData();             //Deliver data
14        Rn = Rn + 1;
15      }
16       SendFrame(Rn);             //Send an ACK
17   }
18 }
```

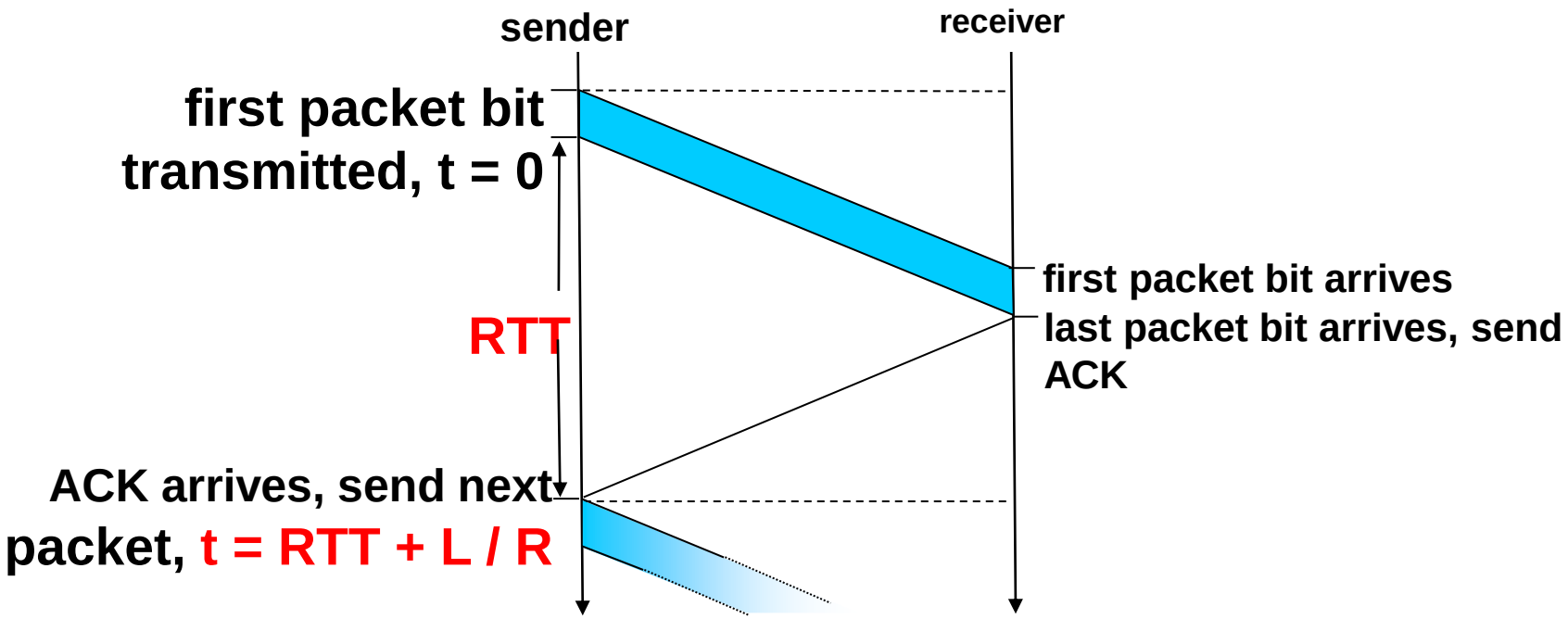
Modulo-2 addition

R_n is the sequence number of the next packet expected

Figure 11.11 *Flow diagram for Example 11.3*



Stop-and-wait operation



L: packet bit length
R: link bandwidth (bps)

$$Utilization = L/R \ / (RTT+L/R)$$

Example 11.4

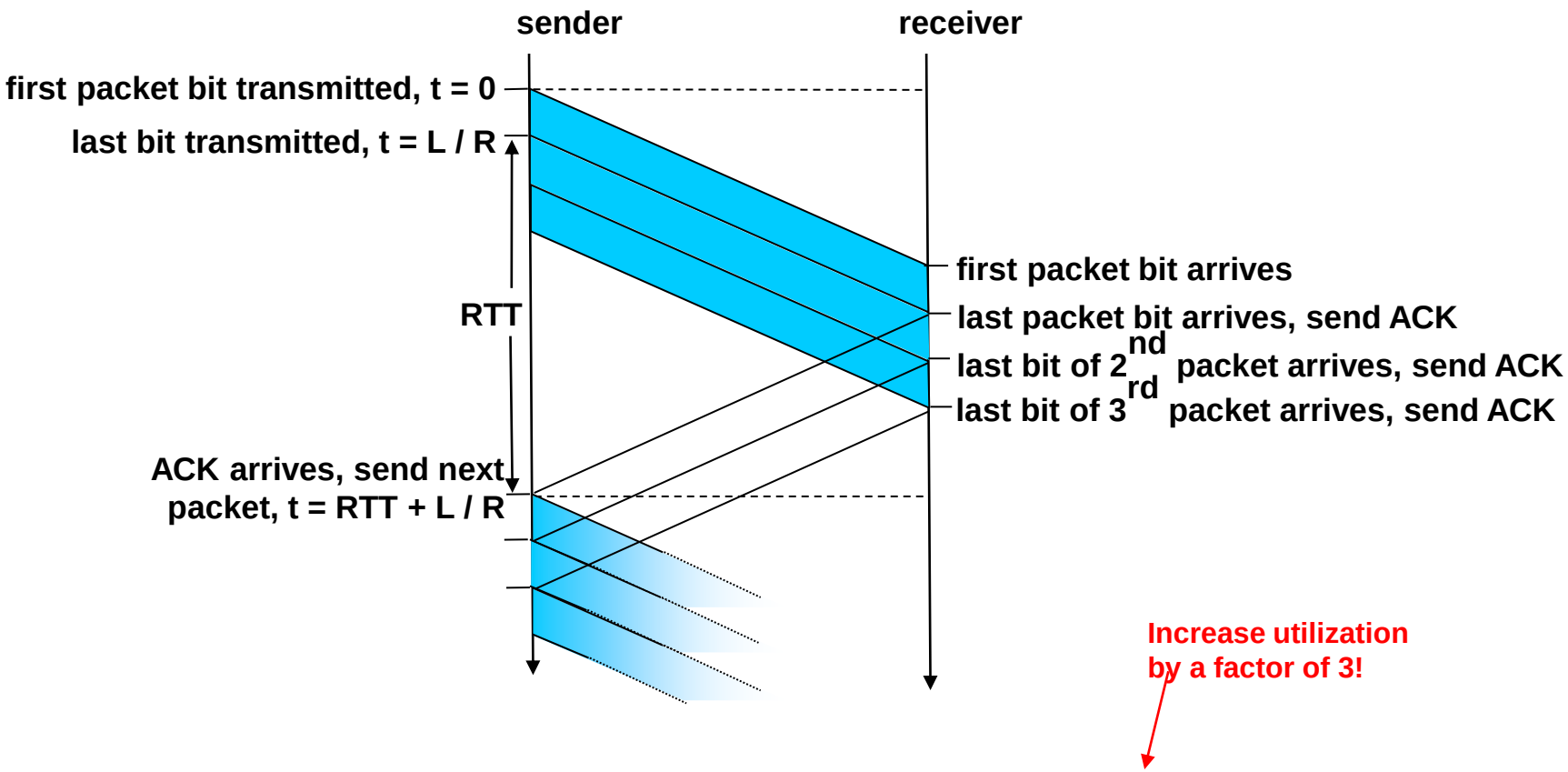
Assume that, in a Stop-and-Wait ARQ system, the bandwidth of the line is 1 Mbps, and 1 bit takes 20 ms to make a round trip. If the system data frames are 1000 bits in length, what is the utilization percentage of the link?

Solution

L = 1000 bits, R = 1Mbps, RTT = 20ms
Utilization = 1/ 21 = 4.8%

For this reason, for a link with a high bandwidth or long delay, the use of Stop-and-Wait ARQ wastes the capacity of the link.

Pipelining: increased utilization



$$Utilization = 3*L/R \ / (RTT+L/R)$$

Example 11.5

What is the utilization percentage of the link in Example 11.4 if we have a protocol that can send up to 15 frames before stopping and worrying about the acknowledgments?

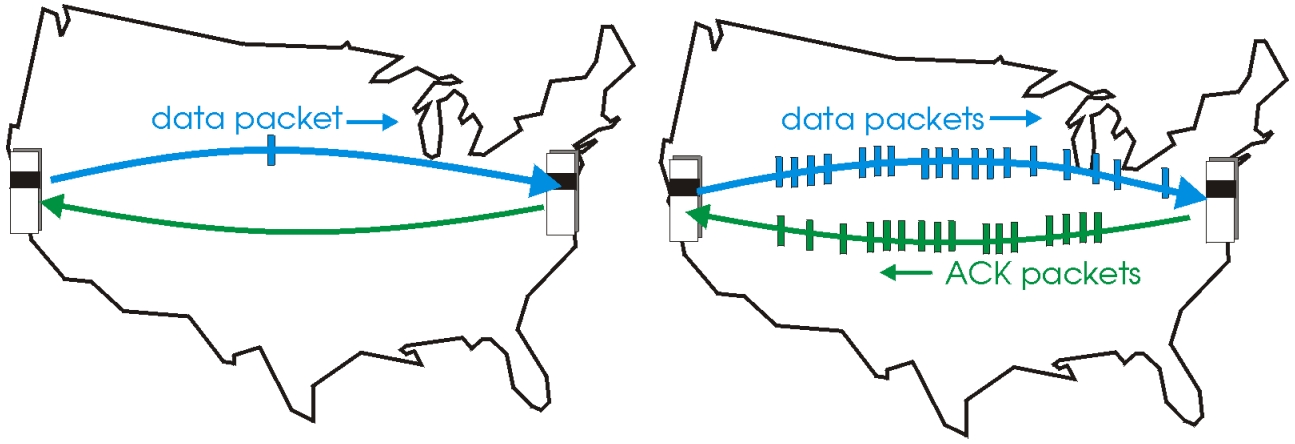
Solution:

*The bandwidth-delay product(R*RTT) is still 20,000 bits. The system can send up to 15 frames or 15,000 bits during a round trip. This means the utilization is 15,000/20,000, or 75 percent. Of course, if there are damaged frames, the utilization percentage is much less because frames have to be resent.*

Pipelined protocols

Pipelining: sender allows multiple, "in-flight", yet-to-be-acknowledged pkts

- range of sequence numbers must be increased
- buffering at sender and/or receiver



(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- Two generic forms of pipelined protocols:
  - *go-Back-N, selective repeat*

Go-Back-N ARQ Protocol

- The key concept of the Go-back-N(GBN) ARQ is to send several packets before receiving acknowledgments.
- The sender keep a copy of the sent packets until the acknowledgments arrive.
- The receiver can only buffer one packet at a time.
- In the Go-Back-N ARQ Protocol, the sequence numbers are modulo 2^m ,range from 0 to 2^m - 1, where m is the size of the sequence number field in bits.
- For example, if m is 4, the only sequence numbers are 0 through 15 inclusive.

0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,0,1,2,3,4,5,6,7,8,9,10,11,...

Figure 11.14 *Design of Go-Back-N ARQ*

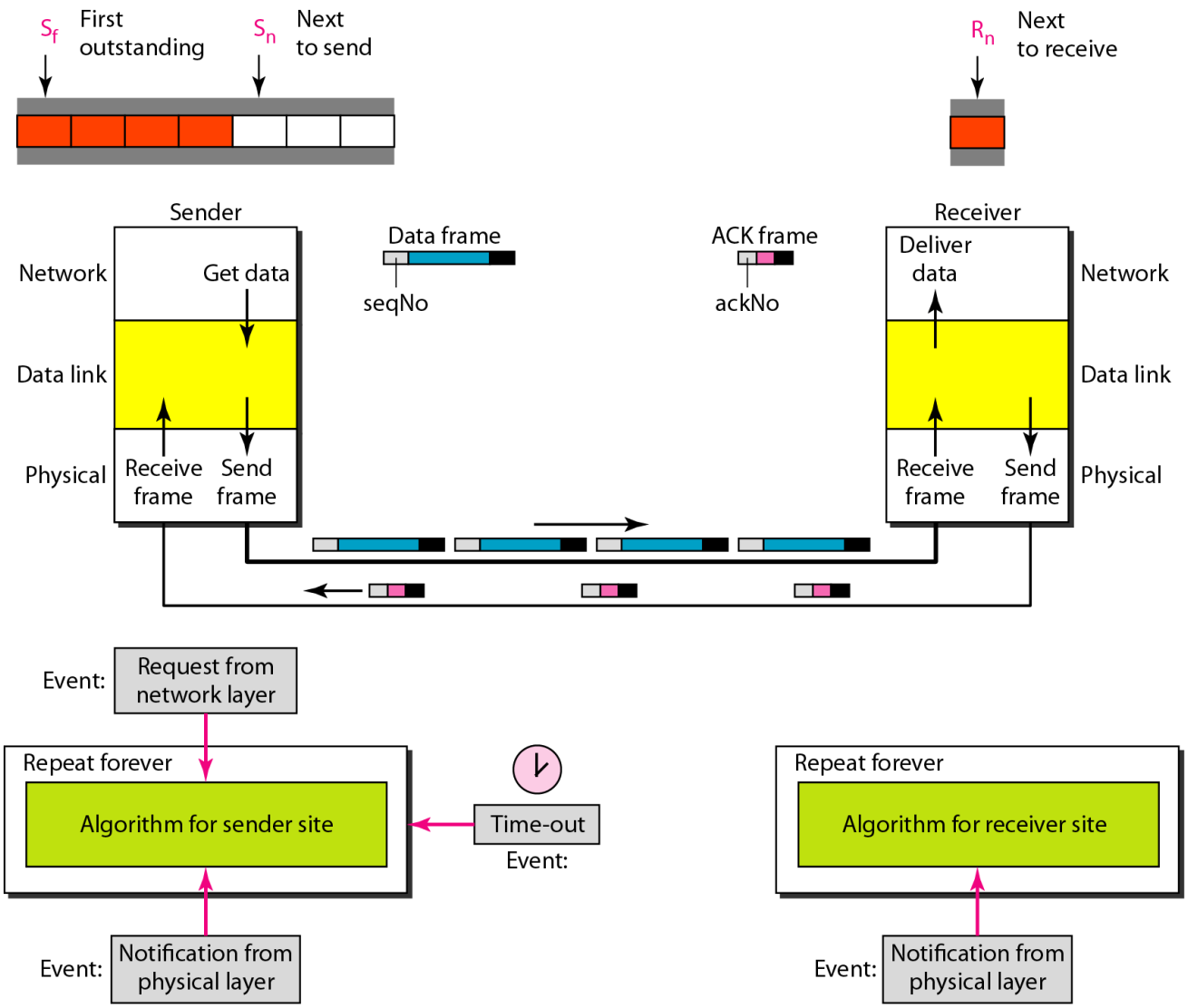
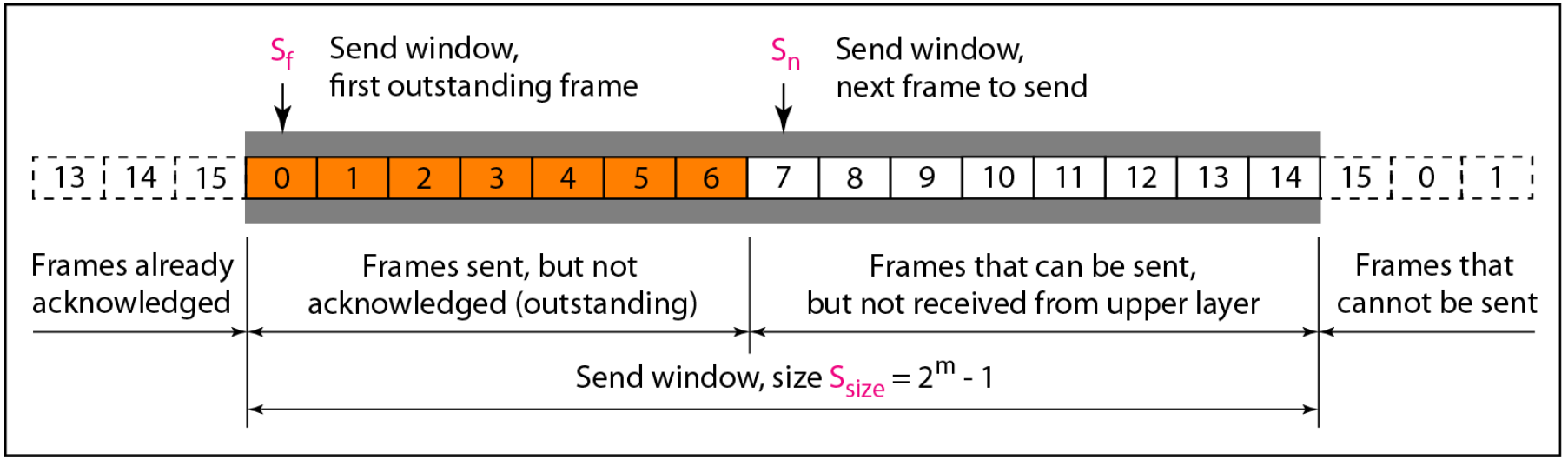
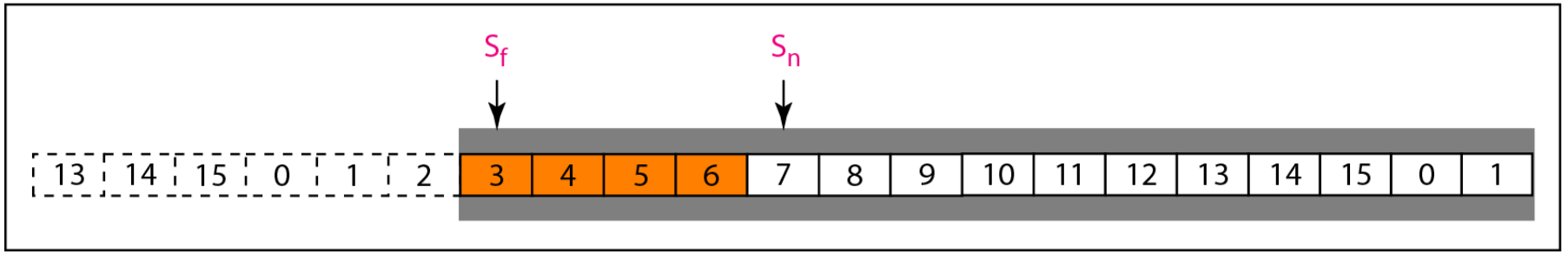


Figure 11.12 *Send window for Go-Back-N ARQ*



a. Send window before sliding



b. Send window after sliding

Note

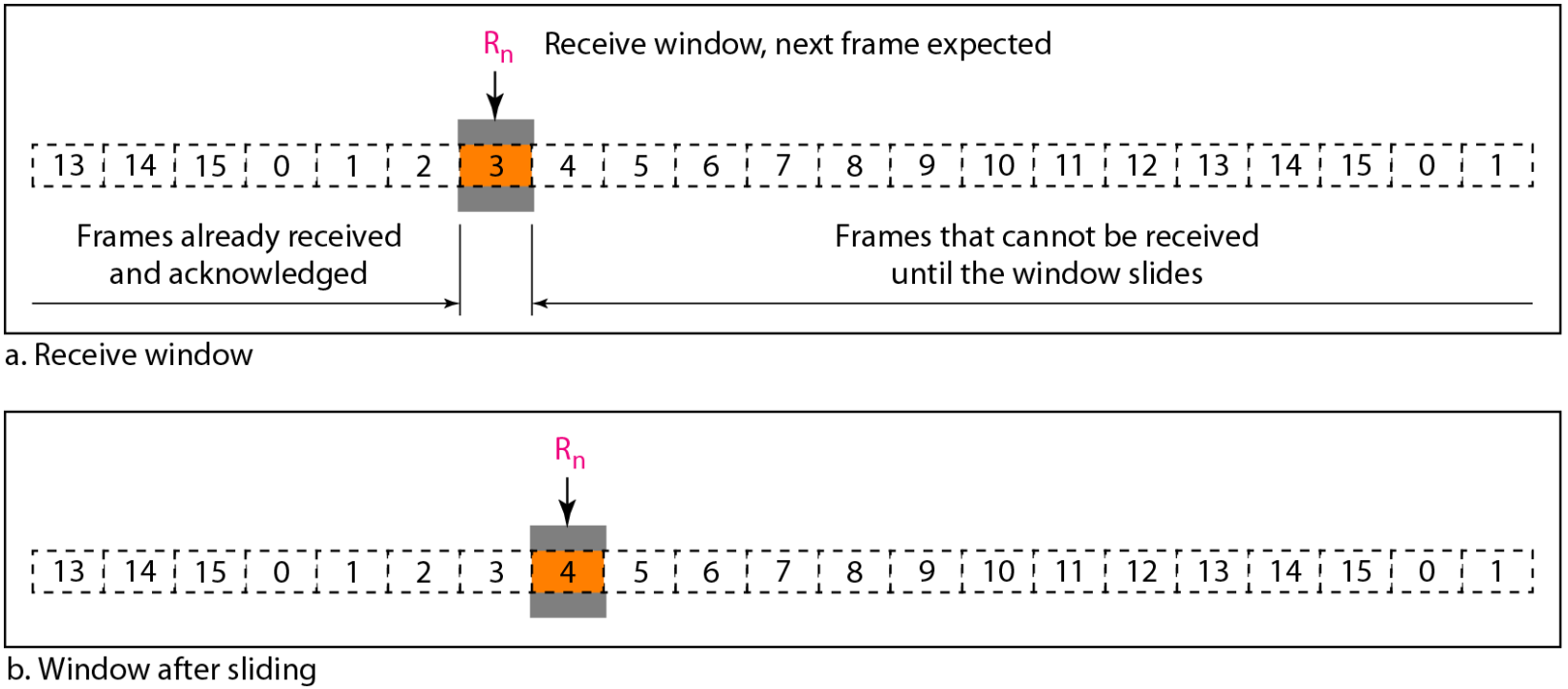
The send window is an abstract concept defining an imaginary box of size $2^m - 1$ with three variables: S_f, S_n, and S_{size}.

The send window can slide one or more slots when a valid acknowledgment arrives.

Cumulative ACK

- **ACK(n): ACKs all pkts up to and include seq # n-1 have been received may receive duplicate ACKs (see receiver)**
- **A single timer for the oldest transmitted but unacked pkt**
- ***timeout:* retransmit all pkts in window (up to N packets)**

Figure 11.13 *Receive window for Go-Back-N ARQ*



Note

The receive window is an abstract concept defining an imaginary box of size 1 with one single variable R_n. The window slides when a correct frame has arrived; sliding occurs one slot at a time.

out-of-order pkt:

discard (don't buffer) -> no receiver buffering!

Re-ACK pkt with highest in-order seq #

Note

Stop-and-Wait ARQ is a special case of Go-Back-N ARQ in which the size of the send window is 1.

Algorithm 11.7 *Go-Back-N sender algorithm*

```
1  Sw = 2^m - 1;
2  Sf = 0;
3  Sn = 0;
4
5  while (true)                          //Repeat forever
6  {
7   WaitForEvent();
8    if(Event(RequestToSend))            //A packet to send
9    {
10      if(Sn-Sf >= Sw)                  //If window is full
11            Sleep();
12      GetData();
13      MakeFrame(Sn);
14      StoreFrame(Sn);
15      SendFrame(Sn);
16      Sn = Sn + 1;
17      if(timer not running)
18           StartTimer();
19   }
20
```

(continued)

Algorithm 11.7 *Go-Back-N sender algorithm* *(continued)*

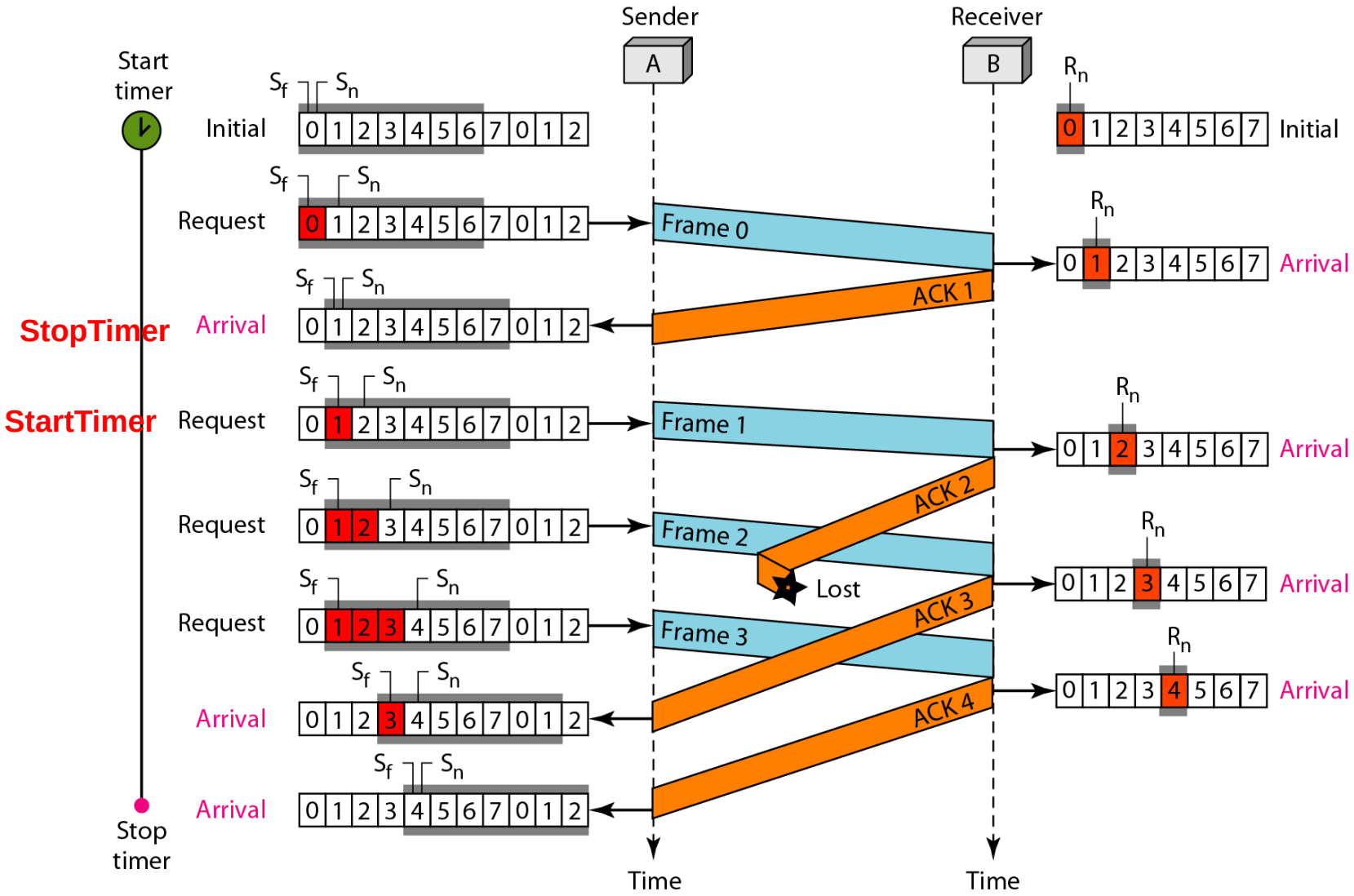
```
21    if(Event(ArrivalNotification))  //ACK arrives
22    {
23       Receive(ACK);
24       if(corrupted(ACK))
25            Sleep();
26       if((ackNo>Sf)&&(ackNo<=Sn)) { //If a valid ACK
27       While(Sf <= ackNo)
28         {
29          PurgeFrame(Sf);
30          Sf = Sf + 1;
31         }
32       StopTimer();
33    }
34
35    if(Event(TimeOut))                    //The timer expires
36    {
37     StartTimer();
38     Temp = Sf;
39     while(Temp < Sn);
40       {
41        SendFrame(Sf);
42        Sf = Sf + 1;
43       }
44    }
45 }
```

If (S_f ==S_n) // the window is empty
StopTimer();
Else
StartTimer();

Algorithm 11.8 *Go-Back-N receiver algorithm*

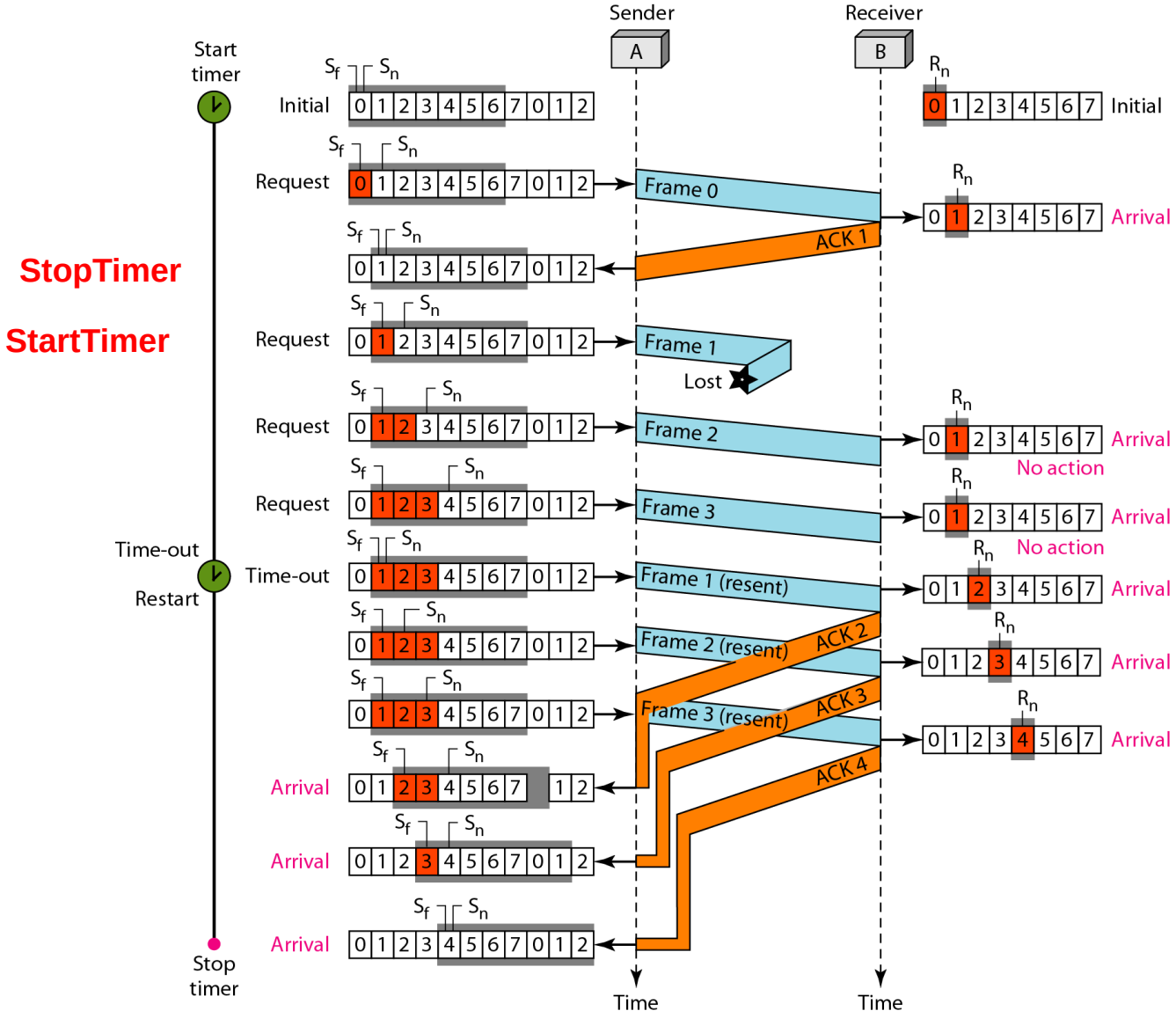
```
1  Rn = 0;
2
3  while (true)                          //Repeat forever
4  {
5    WaitForEvent();
6
7    if(Event(ArrivalNotification)) /Data frame arrives
8    {
9       Receive(Frame);
10      if(corrupted(Frame))
11           Sleep();
12      if(seqNo == Rn)                  //If expected frame
13      {
14        DeliverData();                 //Deliver data
15        Rn = Rn + 1;                   //Slide window
16        SendACK(Rn);
17      }
18   }
19 }
```

Figure 11.16 *Flow diagram for Example 11.6*



Cumulative acknowledgments can help if acknowledgments are delayed or lost

Figure 11.17 *Flow diagram for Example 11.7*



Example 11.7

Figure 11.17 shows what happens when a frame is lost. Frames 0, 1, 2, and 3 are sent. However, frame 1 is lost. The receiver receives frames 2 and 3, but they are discarded because they are received out of order. The sender receives no acknowledgment about frames 1, 2, or 3. Its timer finally expires. The sender sends all outstanding frames (1, 2, and 3) because it does not know what is wrong. Note that the resending of frames 1, 2, and 3 is the response to one single event. When the sender is responding to this event, it cannot accept the triggering of other events. This means that when ACK 2 arrives, the sender is still busy with sending frame 3.

Example 11.7 (continued)

The physical layer must wait until this event is completed and the data link layer goes back to its sleeping state. We have shown a vertical line to indicate the delay. It is the same story with ACK 3; but when ACK 3 arrives, the sender is busy responding to ACK 2. It happens again when ACK 4 arrives. Note that before the second timer expires, all outstanding frames have been sent and the timer is stopped.

Example 11.17 shows that because of one packet lost, all following packets will need to be retransmitted, even if they have arrived at the destination → A great waste of bandwidth

Better protocol: **selective repeat ARQ**

Selective Repeat ARQ

- Problem with Go-back-N:
  - Sender: resend many packets with a single lose
  - Receiver: discard many good received (out-of-order) packets
  - Very inefficient when N becomes bigger (in high-speed network)
- Solution: Receiver *individually* acknowledges all correctly received pkts
  - buffers pkts, as needed, for eventual in-order delivery to upper layer
- sender only resends pkts for which ACK not received
  - sender keeps timer for each unACKed pkt
- sender window
  - N consecutive seq #'s
  - again limits seq #s of sent, unACKed pkts

Figure 11.20 *Design of Selective Repeat ARQ*

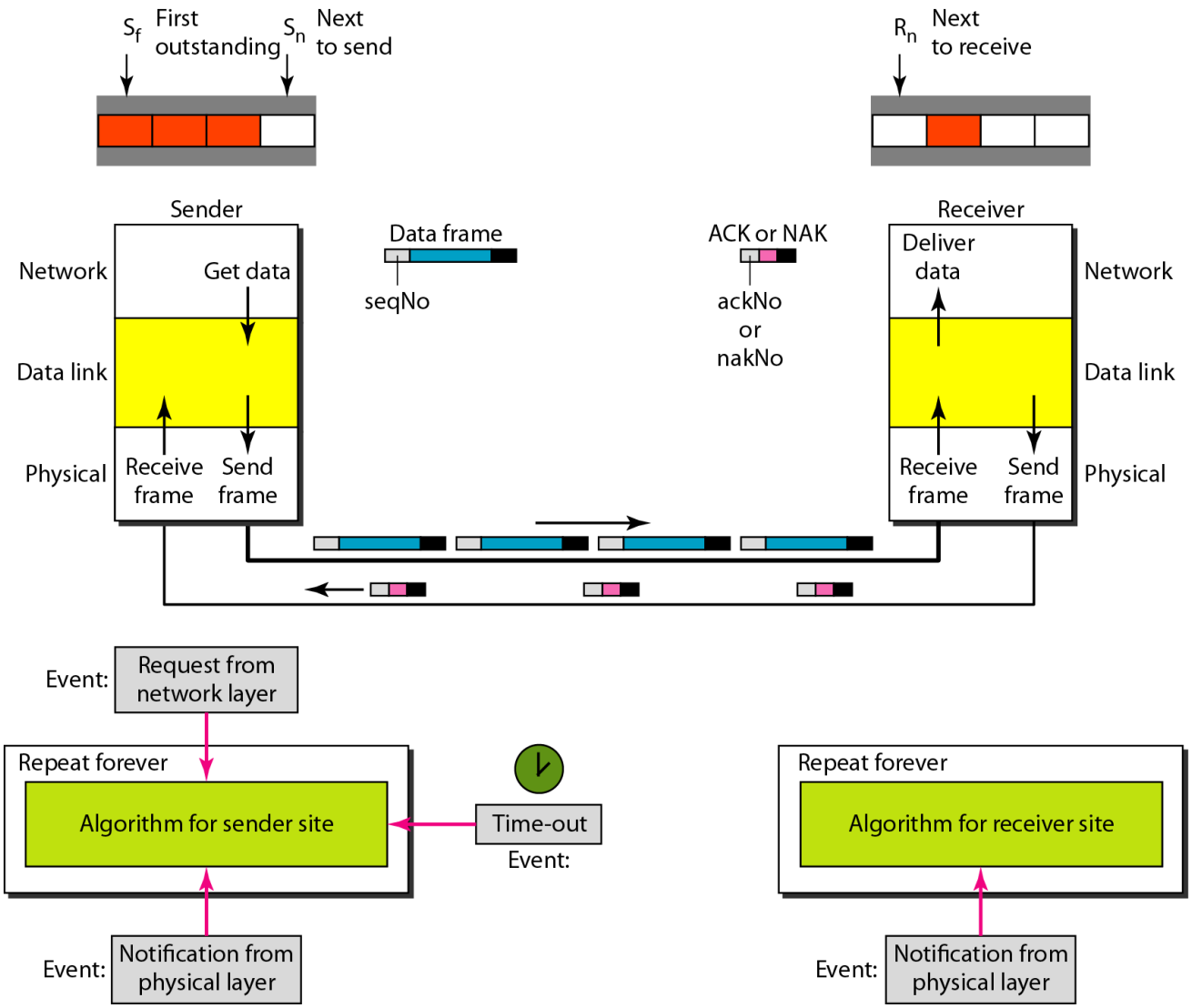


Figure 11.18 *Send window for Selective Repeat ARQ*

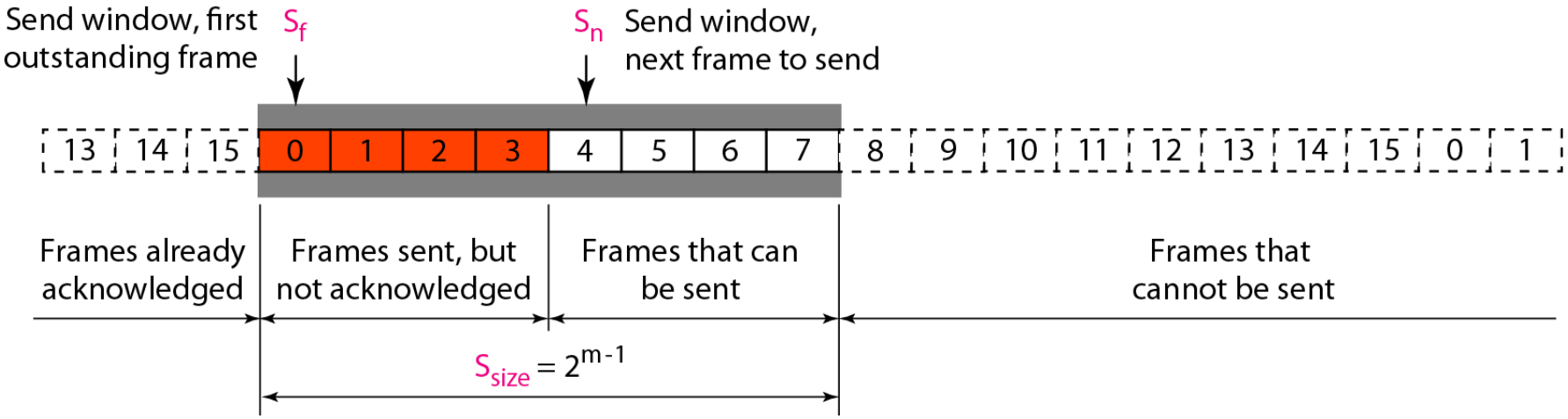


Figure 11.19 *Receive window for Selective Repeat ARQ*

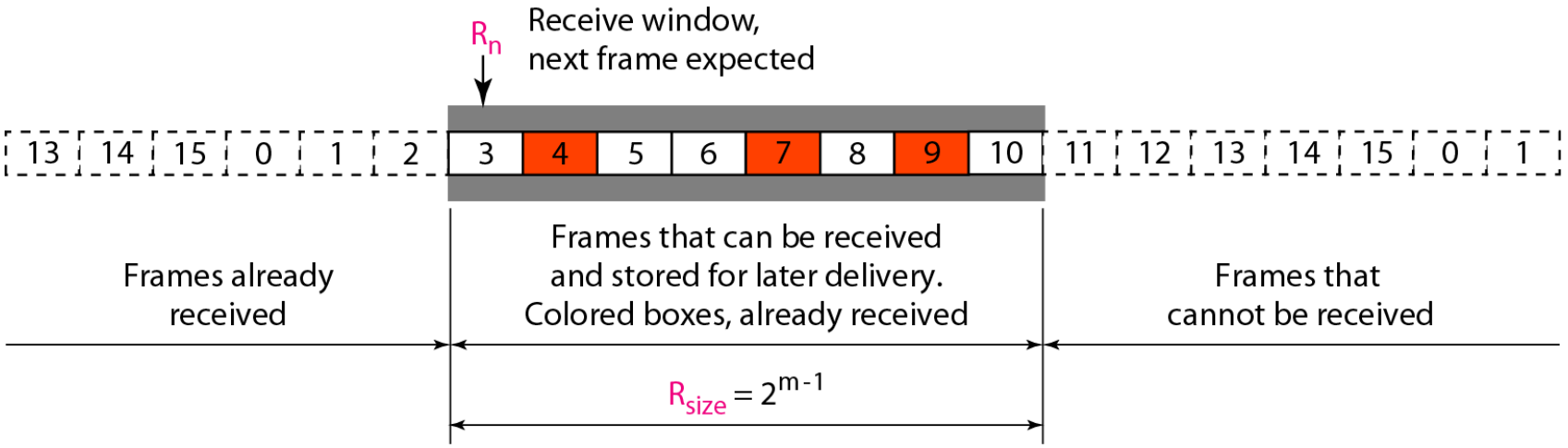
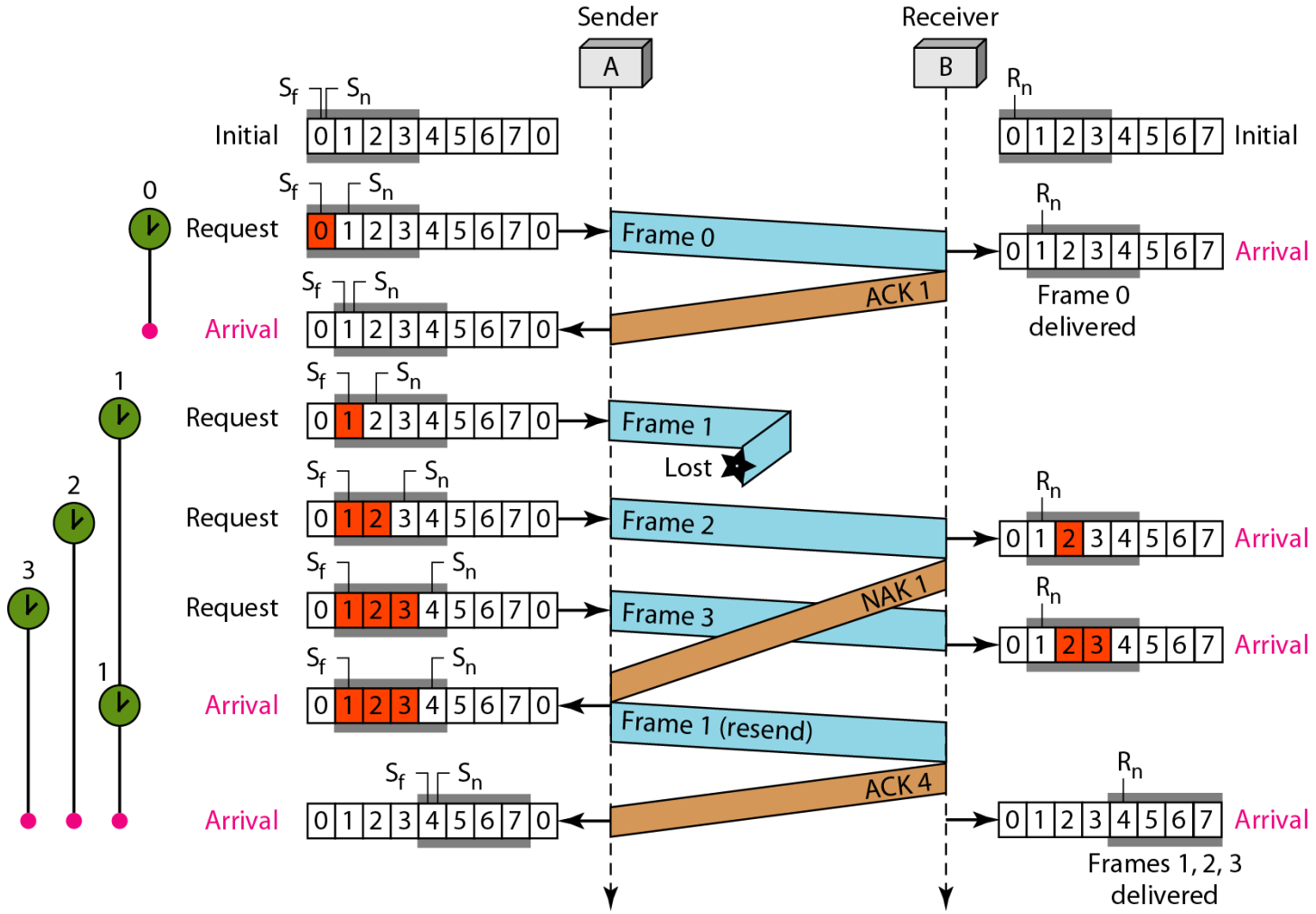


Figure 11.23 *Flow diagram for Example 11.8*



Thank You